Task – 08 SQL Injection Practical Exploitation

Chapter 1 - Introduction to Web Application Security

1.1 Web Application Security

Web applications have become an essential part of modern digital systems, providing services such as online banking, e-commerce, education portals, and social networking platforms. As these applications handle sensitive user data and critical business operations, ensuring their security is extremely important.

Web Application Security refers to the practices, techniques, and measures used to protect web applications from cyber threats, unauthorized access, data breaches, and malicious attacks. Vulnerabilities in web applications can be exploited by attackers to gain access to confidential data, manipulate databases, or disrupt services.

Many security issues arise due to improper input validation, insecure coding practices, weak authentication mechanisms, and misconfigured servers. Among these, **SQL Injection** is one of the most critical and commonly exploited vulnerabilities in web applications.

1.2 Importance of Web Application Security

The importance of web application security can be understood through the following points:

- **Protection of Sensitive Data:**
  Web applications often store personal, financial, and authentication data. Security vulnerabilities can lead to data leakage and identity theft.
- **Maintaining System Integrity:**
  Attacks such as SQL Injection can allow attackers to modify or delete database records, affecting the reliability of the application.
- **Preventing Unauthorized Access:**
  Weak security controls may allow attackers to bypass authentication and gain administrative privileges.
- **Business Continuity:**
  Exploited vulnerabilities can cause application downtime, financial losses, and reputational damage to organizations.
- **Compliance and Legal Requirements:**
  Many organizations must follow security standards and data protection regulations. Failure to secure applications can result in legal penalties.

1.3 Common Web Application Vulnerabilities

Web applications are vulnerable to a wide range of attacks. Some commonly observed vulnerabilities include:

- SQL Injection
- Cross-Site Scripting (XSS)
- Broken Authentication
- Security Misconfiguration
- Insecure Direct Object References
- Sensitive Data Exposure

These vulnerabilities are widely documented by industry standards such as the **OWASP Top 10**, which highlights the most critical security risks faced by web applications.

1.4 Need for Practical Security Testing

While theoretical knowledge of vulnerabilities is important, **practical testing** plays a crucial role in understanding how real-world attacks work. Security testing tools help identify vulnerabilities that may not be visible through code inspection alone.

Practical exploitation using controlled environments allows security professionals and students to:

- Identify vulnerable parameters
- Understand attack techniques
- Observe application behavior during attacks
- Evaluate the impact of vulnerabilities
- Recommend effective mitigation strategies

In this task, a deliberately vulnerable web application environment is used to practically demonstrate SQL Injection attacks using automated tools.

1.5 Objective of This Task

The objective of this task is to gain hands-on experience in identifying and exploiting SQL Injection vulnerabilities in a web application environment. This task focuses on:

- Understanding how SQL Injection vulnerabilities arise
- Using automated tools to exploit vulnerable parameters
- Extracting database information securely in a controlled setup
- Analyzing the impact of such vulnerabilities
- Learning appropriate mitigation and prevention techniques

This practical approach strengthens the understanding of web application security concepts and prepares learners for real-world security assessment scenarios.

Chapter 2 - Overview of SQL Injection

2.1 Introduction to SQL Injection

SQL Injection (SQLi) is a critical web application vulnerability that occurs when user-supplied input is improperly handled and directly included in SQL queries. Attackers can manipulate these inputs to alter the intended SQL query logic, allowing unauthorized access to the backend database.

SQL Injection attacks exploit insecure coding practices where applications fail to validate or sanitize input before executing database queries. As a result, attackers can retrieve sensitive information, modify database records, or even gain full control over the database server.

Due to its severe impact and high prevalence, SQL Injection is consistently ranked as a top security risk in the OWASP Top 10 list.

2.2 How SQL Injection Works

In a vulnerable web application, user input is often used to dynamically construct SQL queries. When this input is not properly validated, an attacker can inject malicious SQL statements that change the behavior of the original query.

For example, instead of submitting a normal numeric input, an attacker may insert SQL operators or conditions that force the database to return unintended results. The database executes the modified query without distinguishing between legitimate input and injected commands.

This enables attackers to:

- Bypass authentication mechanisms
- Extract database contents
- Enumerate database structure
- Perform administrative operations

2.3 Types of SQL Injection

SQL Injection attacks can be categorized into several types based on how information is retrieved from the database.

2.3.1 In-band SQL Injection

In-band SQL Injection is the most common type, where the attacker receives the results of the injection directly through the same communication channel.

- **Error-based SQL Injection:**
  Database error messages are exploited to obtain information about the database structure and queries.

- **Union-based SQL Injection:**
  The UNION SQL operator is used to combine the results of the original query with attacker-controlled queries, allowing direct data extraction.

2.3.2 Blind SQL Injection

Blind SQL Injection occurs when the application does not display database errors or query results directly. Instead, attackers infer information based on application behavior.

- **Boolean-based Blind SQL Injection:**
  The application's response changes based on true or false conditions injected into the query.
- **Time-based Blind SQL Injection:**
  Database delays (such as time delays) are used to determine whether injected conditions are true or false.

This type of SQL Injection is slower but highly effective when error messages are suppressed.

2.3.3 Out-of-band SQL Injection

Out-of-band SQL Injection relies on external channels, such as DNS or HTTP requests, to extract data. This method is used when direct responses are not available and the database supports external communication.

Although less common, it can be extremely powerful in certain environments.

2.4 Impact of SQL Injection Attacks

Successful SQL Injection attacks can have severe consequences, including:

- Unauthorized access to sensitive user data
- Exposure of usernames, passwords, and personal information
- Modification or deletion of database records
- Complete compromise of backend databases
- Loss of data integrity and confidentiality
- Legal, financial, and reputational damage to organizations

2.5 SQL Injection in Practical Scenarios

SQL Injection vulnerabilities are frequently found in:

- Login forms
- Search fields
- URL parameters

- Cookies and session identifiers

In practical security testing, automated tools such as SQLMap are commonly used to identify and exploit SQL Injection vulnerabilities efficiently. These tools help automate payload generation, vulnerability detection, and data extraction.

Chapter 3 - Lab Setup and Testing Environment

3.1 Purpose of the Lab Environment

To understand and demonstrate SQL Injection attacks in a safe and ethical manner, a controlled testing environment is required. Performing such attacks on real-world applications without authorization is illegal and unethical. Therefore, a deliberately vulnerable application is used to simulate real-world scenarios without causing harm.

For this task, **Damn Vulnerable Web Application (DVWA)** was used as the target application, and **SQLMap** was used as the exploitation tool. This setup allows hands-on learning of SQL Injection techniques, database enumeration, and data extraction.

3.2 Overview of Damn Vulnerable Web Application (DVWA)

Damn Vulnerable Web Application (DVWA) is a PHP/MySQL-based web application intentionally designed with multiple security vulnerabilities. It is widely used for educational and training purposes in web application security.

DVWA provides:

- Multiple vulnerability categories
- Adjustable security levels
- Realistic exploitation scenarios

In this task, the **SQL Injection** vulnerability module of DVWA was used.

3.3 System Requirements

The testing environment was set up using the following components:

- **Operating System:** Kali Linux (Virtual Machine)
- **Web Server:** Apache
- **Database Server:** MariaDB (MySQL-compatible)
- **Vulnerable Application:** DVWA
- **Exploitation Tool:** SQLMap

All components were installed and executed locally to ensure a secure testing environment.

3.4 Starting Required Services

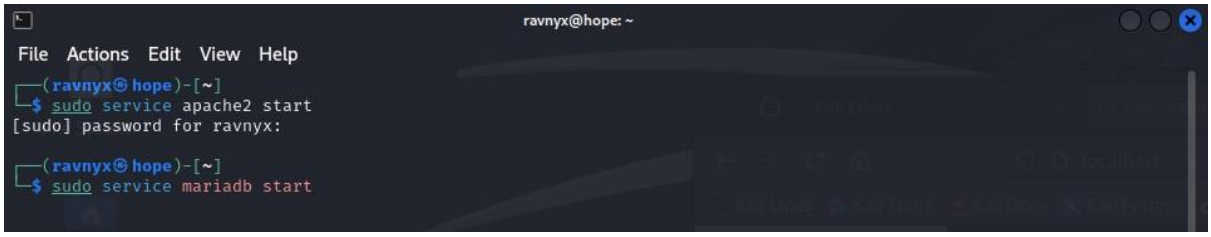
Before launching DVWA, the required backend services were started to ensure proper functioning of the application.

The following services were enabled:

- Apache web server
- MariaDB database server

These services allow DVWA to communicate with the database and process user requests.

Figure 3.4: Apache and MariaDB services successfully started to support DVWA operation.



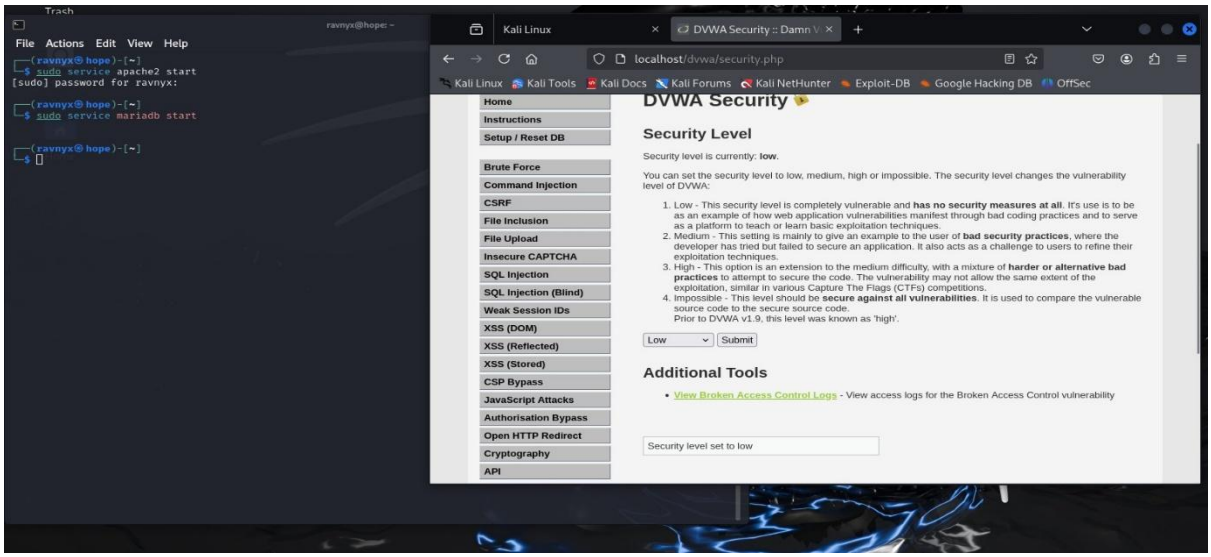
3.5 DVWA Configuration

After launching DVWA in the browser, the application was configured as follows:

- **Security Level:** Low
- **Database:** Successfully set up using the “Setup / Reset DB” option

Setting the security level to *Low* disables input validation and protection mechanisms, making the application intentionally vulnerable to SQL Injection attacks. This is essential for observing exploitation behavior clearly during testing.

Figure 3.5: DVWA security level configured to Low to allow SQL Injection testing.



3.6 Identification of Vulnerable Parameter

Within DVWA, the **SQL Injection** module was selected. The vulnerable input field accepts a **User ID**, which is passed as a URL parameter (id) using the GET method.

This parameter was identified as injectable due to:

- Direct interaction with the backend database
- Lack of input sanitization at low security level
- Observable abnormal behavior when SQL payloads were inserted

3.7 Introduction to SQLMap

SQLMap is an open-source automated SQL Injection tool used to detect and exploit SQL Injection vulnerabilities. It supports various database management systems and provides features such as:

- Automated vulnerability detection
- Database enumeration
- Table and column extraction
- Data dumping
- Password hash identification and cracking

SQLMap was used in this task to automate the exploitation of the identified SQL Injection vulnerability in DVWA.

3.8 Ethical Considerations

All testing performed in this lab was conducted on a local, intentionally vulnerable application. No real-world systems or user data were targeted. This ensures compliance with ethical hacking principles and legal guidelines.

Chapter 4 - SQL Injection Exploitation and Database Enumeration

4.1 Introduction to SQL Injection

SQL Injection is a web application vulnerability that allows an attacker to interfere with the queries an application makes to its database. This occurs when user-supplied input is directly included in SQL queries without proper validation or sanitization.

In this task, SQL Injection was exploited to:

- Identify vulnerable parameters
- Enumerate backend databases
- Extract tables and sensitive user data

The vulnerability was tested using **SQLMap** against the SQL Injection module of DVWA.

4.2 Identification of Injectable Parameter

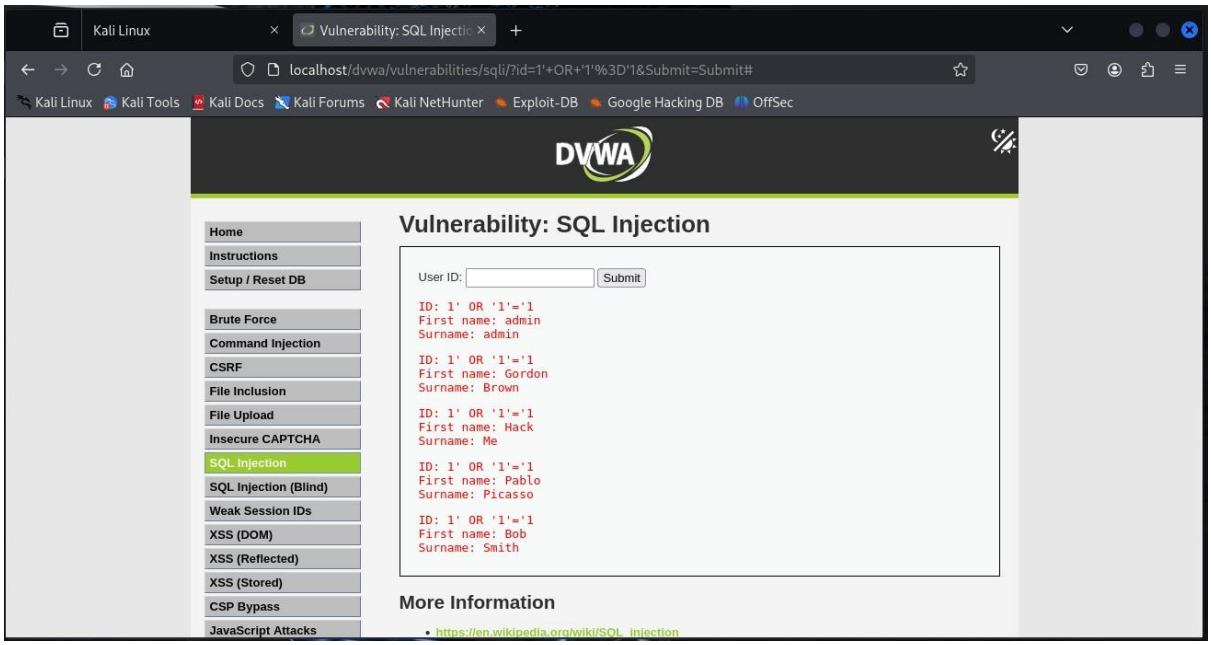
During testing, the **id** parameter in the SQL Injection module of DVWA was identified as vulnerable.

Key observations:

- The parameter is passed through the URL using the **GET** method
- Input values are directly processed by the backend database
- Manual payloads such as logical conditions (1 OR 1=1) returned abnormal results

This confirmed that the ***id*** parameter was injectable and suitable for automated exploitation using SQLMap.

Figure 4.2: Manual SQL Injection test confirming the id parameter is vulnerable.



4.3 Running SQLMap for Vulnerability Detection

After identifying the injectable parameter, SQLMap was executed to verify the vulnerability and determine the backend database type.

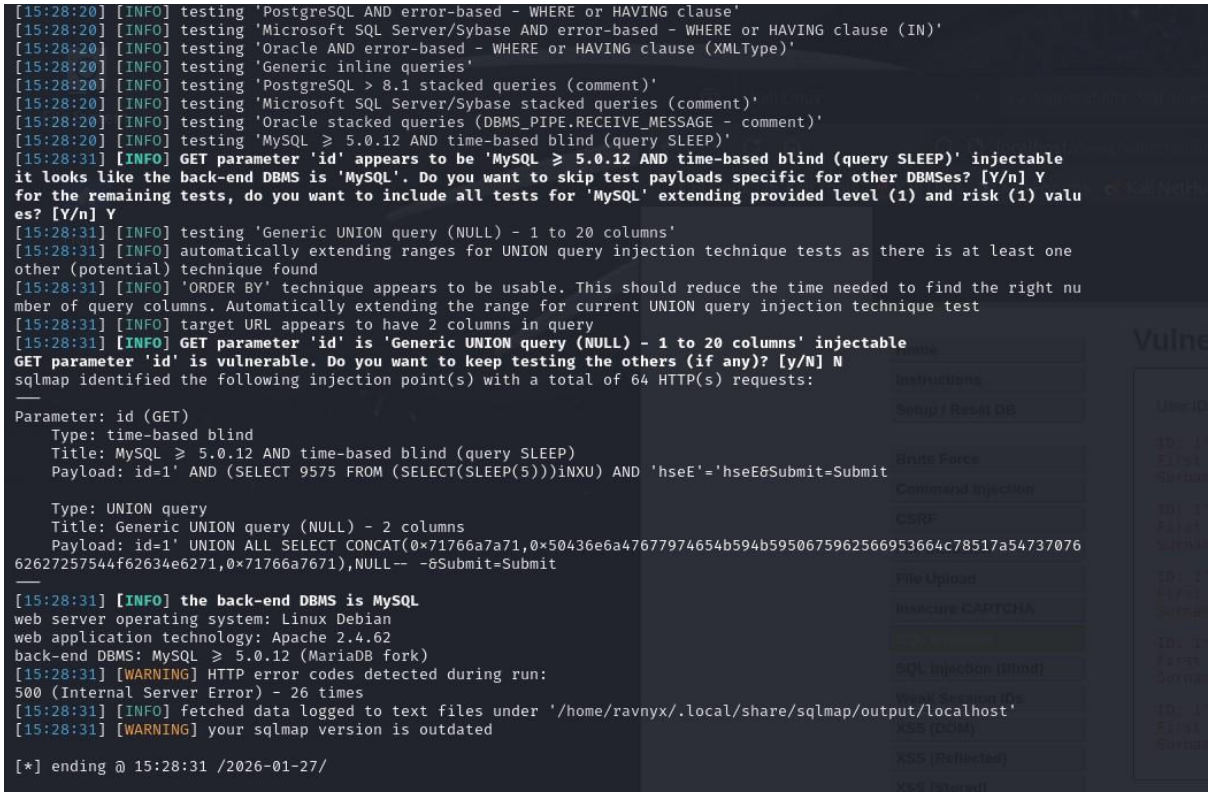
SQLMap successfully:

- Detected SQL Injection vulnerability
- Identified the injection technique as **time-based blind** and **UNION-based**
- Determined the backend database as **MySQL (MariaDB fork)**

This confirms that the application is vulnerable to SQL Injection attacks.

Figure 4.3: SQLMap executed against the vulnerable id parameter using authenticated session cookies.

Figure 4.3: SQLMap confirming SQL Injection vulnerability on the id parameter and identifying MySQL as the backend DBMS.



4.4 Database Enumeration

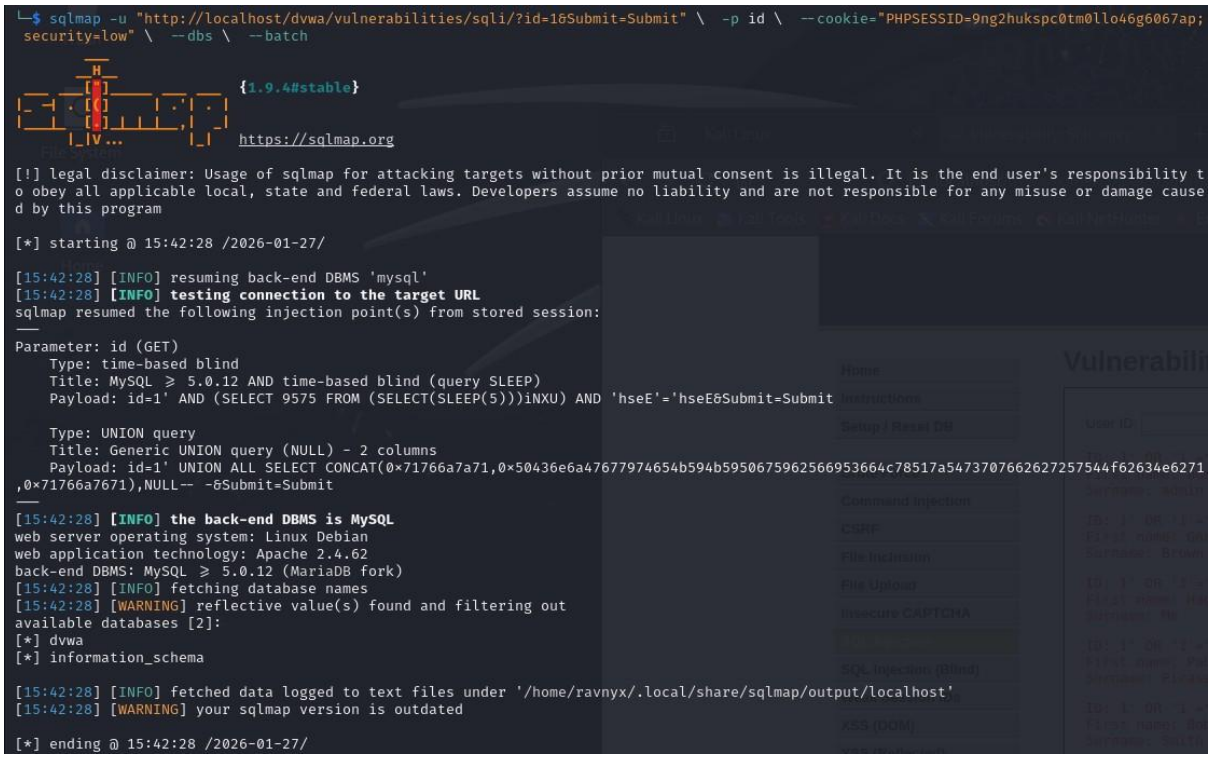
Once SQL Injection was confirmed, SQLMap was used to enumerate available databases on the target system.

SQLMap extracted the following database names:

- **dvwa**
- **information_schema**

The presence of the **dvwa** database indicates the main application database, while **information_schema** is a system database used by MySQL to store metadata.

Figure 4.4: SQLMap enumerating accessible databases from the vulnerable application.



4.5 Table Enumeration from Target Database

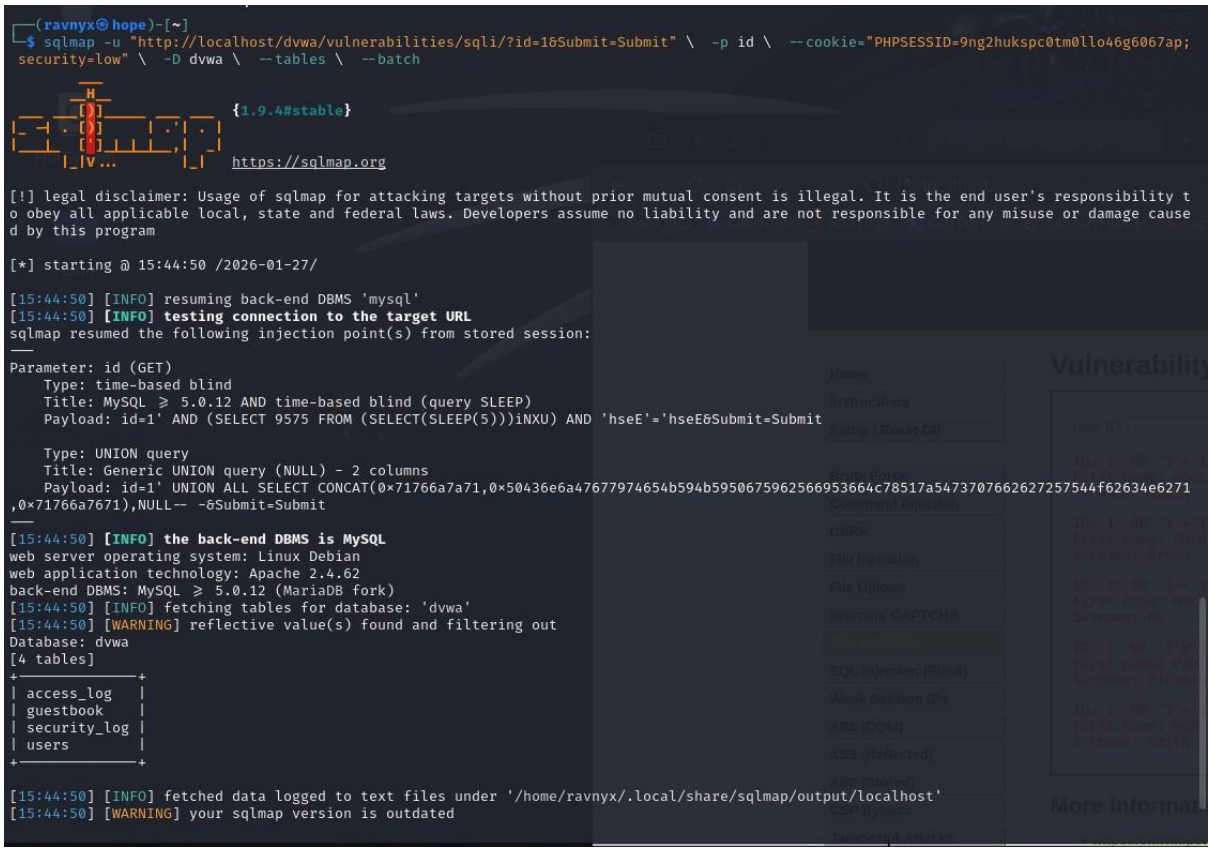
After identifying the target database (dvwa), SQLMap was used to extract table names.

The following tables were discovered:

- **users**
- **guestbook**
- **security_log**
- **access_log**

These tables store application-specific data and logs, with the **users** table containing sensitive authentication information.

Figure 4.5: SQLMap extracting table names from the DVWA database.



4.6 Significance of Enumeration Results

The successful enumeration of databases and tables demonstrates the severity of SQL Injection vulnerabilities. An attacker can:

- Gain full visibility of database structure
- Identify sensitive tables
- Prepare for further data extraction

This stage forms the foundation for deeper exploitation, such as dumping user credentials.

4.7 Summary of Chapter

In this chapter:

- The vulnerable id parameter was identified
- SQLMap confirmed SQL Injection vulnerability
- Backend DBMS was identified as MySQL
- Databases and tables were successfully enumerated

This completes the **database discovery phase** of the SQL Injection attack.

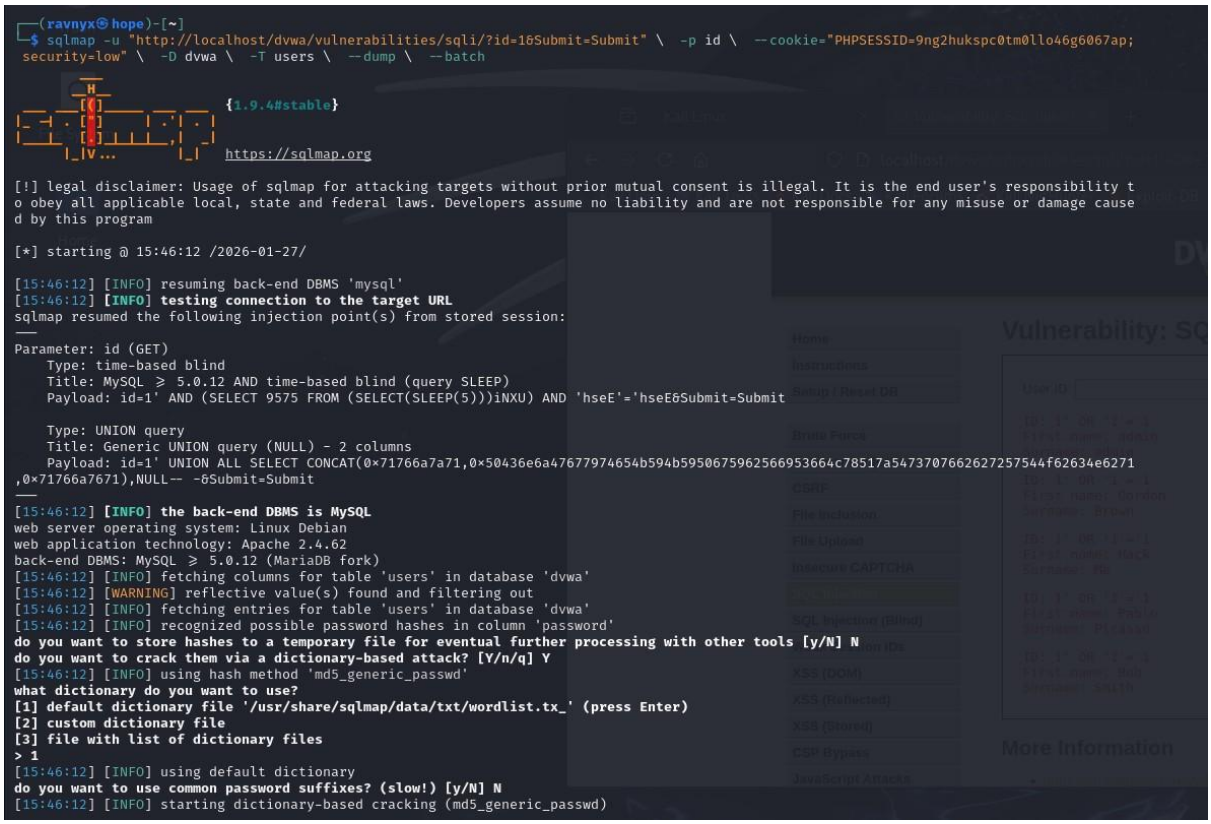
Chapter 5 - Extraction of User Data and Credential Analysis

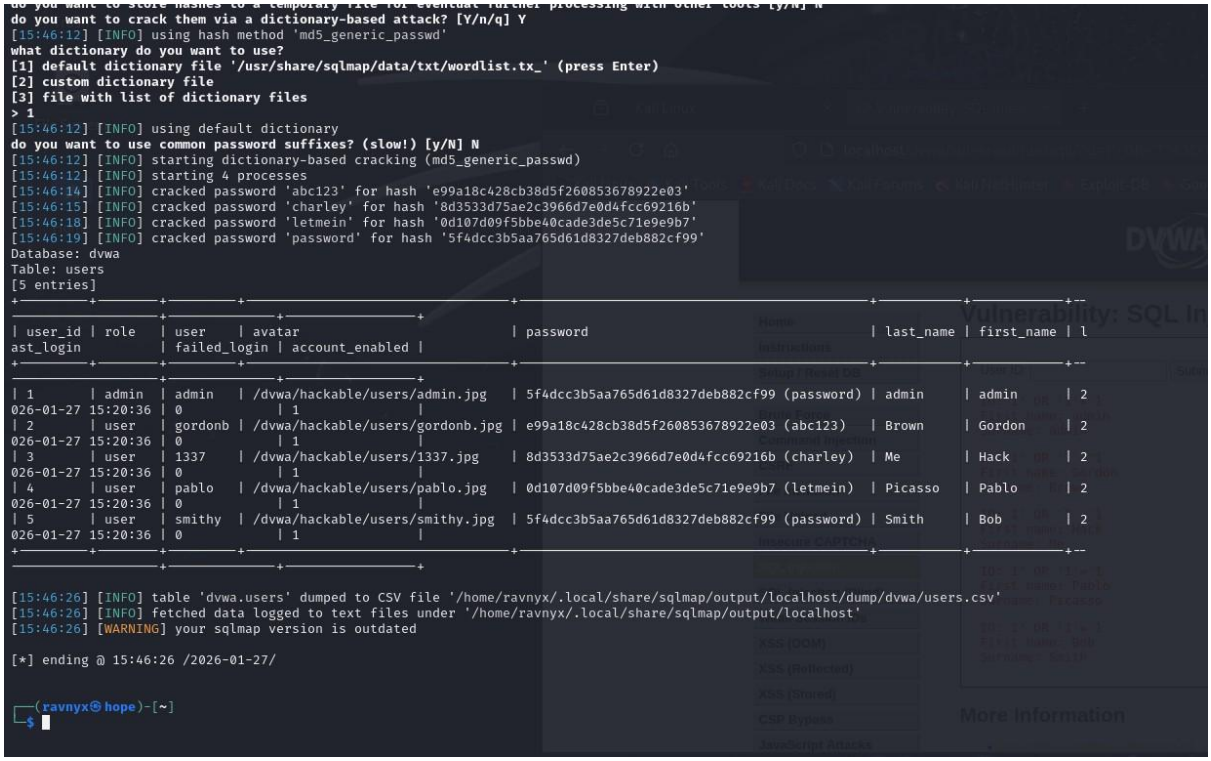
5.1 User Data Extraction Using SQLMap

After successful database and table enumeration, SQLMap was used to extract sensitive data from the **users** table of the DVWA database. This table stores authentication-related information such as usernames and passwords.

Using automated SQL Injection exploitation, SQLMap retrieved the contents of the users table without requiring any valid authorization beyond the existing session. This demonstrates how attackers can abuse SQL Injection vulnerabilities to directly access confidential information stored in backend databases.

Figure 5.1: SQLMap dumping sensitive user credentials from the DVWA database.





5.2 Retrieved User Information

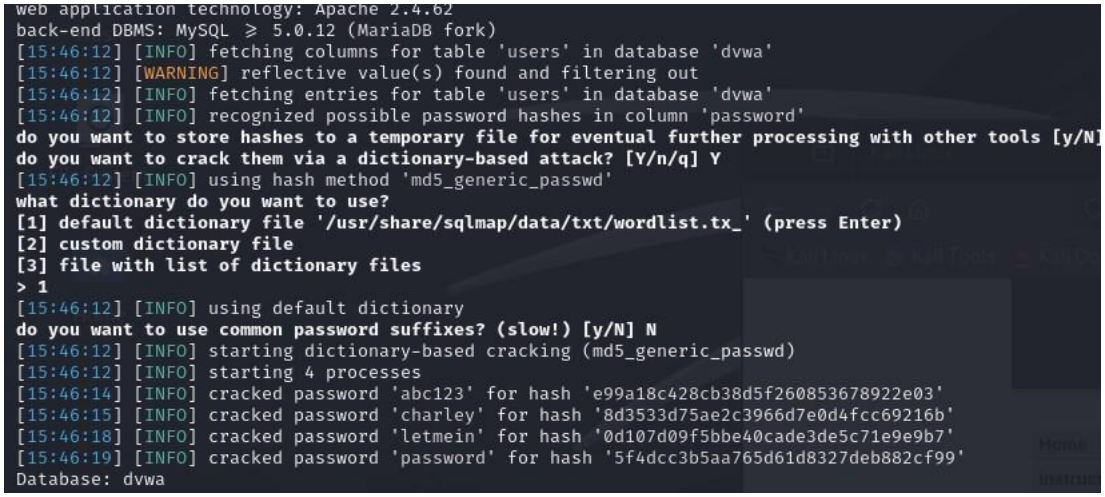
The extracted data from the users table included:

- Usernames
- Password hashes
- Associated user identifiers

In some cases, SQLMap was able to crack weak password hashes using built-in dictionary-based techniques. This exposed plaintext passwords for several user accounts, including administrative-level users.

This highlights the danger of storing passwords using weak hashing algorithms or without additional security mechanisms such as salting.

Figure 5.2: SQLMap successfully cracking weak password hashes using dictionary-based techniques.



5.3 Credential Exposure and Security Implications

The exposure of user credentials represents a critical security risk. If such an attack were performed on a real-world application, attackers could:

- Gain unauthorized access to user accounts
- Escalate privileges by compromising administrative users
- Reuse credentials on other platforms (credential stuffing)
- Perform identity theft and impersonation
- Modify or delete sensitive application data

The compromise of authentication data significantly increases the attack surface of an application and can lead to complete system takeover.

5.4 Analysis of Attack Impact

The successful extraction of user credentials demonstrates the severe impact of SQL Injection vulnerabilities. The attack required no direct access to the database server and was performed entirely through the web application interface.

This confirms that:

- Input validation was insufficient
- Database queries were dynamically constructed
- Sensitive data was accessible through exploitable parameters

Such vulnerabilities can cause irreparable damage to data confidentiality, integrity, and availability.

5.5 Summary of Chapter

In this chapter:

- Sensitive user data was successfully extracted from the database

- Password hashes and user credentials were exposed
- The real-world consequences of SQL Injection attacks were analyzed

This stage represents the **most critical phase** of SQL Injection exploitation, clearly demonstrating why such vulnerabilities must be prevented in production systems.

Chapter 6 - Mitigation and Prevention Techniques

6.1 Input Validation and Sanitization

One of the primary causes of SQL Injection vulnerabilities is improper input handling. All user-supplied input should be strictly validated and sanitized before being processed by the application. Input validation ensures that only expected data types and formats are accepted, reducing the risk of malicious payloads being injected into SQL queries.

6.2 Use of Prepared Statements (Parameterized Queries)

Prepared statements are one of the most effective defenses against SQL Injection attacks. Instead of dynamically constructing SQL queries using user input, prepared statements separate SQL logic from data. This ensures that user input is treated strictly as data and not as executable SQL code.

Using parameterized queries prevents attackers from modifying query structure, even if malicious input is provided.

6.3 Least Privilege Principle

Database users should be granted only the minimum permissions required to perform their tasks. Applications should not use database accounts with administrative privileges unless necessary. By limiting database permissions, the impact of a successful SQL Injection attack can be significantly reduced.

6.4 Secure Error Handling

Displaying detailed database error messages to users can provide attackers with valuable information about database structure and query logic. Applications should implement secure error handling mechanisms that log detailed errors internally while displaying generic error messages to users.

This reduces the risk of information leakage that can aid SQL Injection attacks.

6.5 Use of Web Application Firewalls (WAF)

Web Application Firewalls can help detect and block common SQL Injection payloads before they reach the application. While WAFs should not replace secure coding practices, they serve as an additional layer of defense by filtering malicious requests and monitoring suspicious activity.

6.6 Regular Security Testing

Regular security assessments such as vulnerability scanning, penetration testing, and code reviews help identify SQL Injection vulnerabilities early in the development lifecycle. Automated tools, combined with manual testing, provide better coverage and help maintain application security.

6.7 Secure Password Storage

Passwords should be stored using strong cryptographic hashing algorithms with proper salting. Weak or outdated hashing mechanisms increase the risk of credential compromise if attackers gain database access. Secure password storage reduces the impact of SQL Injection-related data breaches.

6.8 Summary of Chapter

This chapter discussed various mitigation and prevention techniques to protect web applications from SQL Injection attacks. Implementing secure coding practices, enforcing least privilege, and conducting regular security testing are essential steps in building resilient web applications.

Chapter 7 - Conclusion

This task successfully demonstrated the practical exploitation of **SQL Injection** vulnerabilities using a controlled and intentionally vulnerable web application environment. By using DVWA and SQLMap, the complete attack lifecycle—from identifying injectable parameters to extracting sensitive user data—was performed in a safe and ethical manner.

The task highlighted how improper input validation and insecure query construction can lead to severe security risks, including unauthorized database access, exposure of user credentials, and potential system compromise. The automated exploitation using SQLMap further emphasized how easily attackers can take advantage of such vulnerabilities when security controls are weak.

Through hands-on testing, this task strengthened the understanding of SQL Injection attack techniques, database enumeration methods, and the real-world impact of compromised authentication data. Additionally, the mitigation strategies discussed reinforce the importance of secure coding practices, proper database configuration, and regular security testing.

Overall, this task provided valuable practical experience in web application security testing and emphasized the critical need for proactive vulnerability assessment and secure development practices to protect modern web applications.